

Ordenação em Memória Secundária

Integrantes: Pedro Brassi Lucas (2022.2.08.009), Gustavo Silva Orlando (2024.2.08.009).

Professor: Iago Augusto de Carvalho.

Disciplina: Algoritmo e Estrutura de Dados II.

Local: Universidade Federal de Alfenas – UNIFAL.

Data: 5 de novembro de 2025.

1.Introdução

A ordenação em memória secundária é uma técnica essencial para lidar com grandes volumes de dados que excedem a capacidade da memória principal (RAM). Em muitos contextos, como bancos de dados, análise de grandes arquivos de log e processamento de informações em larga escala, é comum encontrar arquivos que não podem ser completamente carregados na memória para serem ordenados de forma convencional.

Diante dessa limitação, os algoritmos de ordenação externa (ou “External Sorting”) foram desenvolvidos para permitir que a ordenação ocorra parcialmente na memória e majoritariamente em dispositivos de armazenamento secundário, como discos rígidos ou SSDs.

Este trabalho tem como objetivo implementar e analisar um algoritmo de ordenação em memória secundária que seja capaz de processar um arquivo de inteiros cujo tamanho ultrapasse 1,5 vezes a capacidade de memória RAM do computador utilizado. O código deve ser capaz de gerar o arquivo de entrada, realizar a ordenação eficiente desses dados e gerar um relatório final com informações sobre o tempo total de execução e os recursos utilizados.

2. Estruturas de Dados Utilizadas

Para a implementação deste projeto, foram utilizadas as seguintes estruturas:

2.1. Vetores Dinâmicos (int *array):

Responsáveis por armazenar temporariamente os dados de cada bloco lido do arquivo principal. Esses vetores são alocados dinamicamente de acordo com o tamanho do bloco processado (aproximadamente 1 GB).

2.2. Arquivos Binários (FILE *):

Usados para leitura e escrita dos dados diretamente em disco, de forma a reduzir o uso de memória RAM. Essa abordagem permite processar arquivos que podem atingir dezenas de gigabytes.

2.3. Arquivos Temporários:

Após a leitura de cada bloco, os dados são ordenados e gravados em arquivos temporários, que são posteriormente mesclados na etapa final de intercalação. Essas estruturas garantem eficiência e estabilidade mesmo com grandes volumes de dados, mantendo o uso de memória sob controle.

3. Descrição do Algoritmo

O algoritmo escolhido para este trabalho foi o **Merge Sort Externo**, uma variação do algoritmo clássico de merge sort adaptada para trabalhar com dados armazenados em disco.

3.1. Etapas do algoritmo:

- Geração dos dados:

O programa solicita ao usuário o tamanho desejado do arquivo de entrada (em gigabytes) e gera números inteiros aleatórios até preencher esse tamanho.

O arquivo é salvo em disco e servirá como entrada para a ordenação.

- Divisão em blocos:

O arquivo é lido e dividido em blocos de até 1 GB. Cada bloco é carregado individualmente na memória principal, ordenado localmente e salvo como um arquivo temporário.

- Ordenação dos blocos:

Cada bloco é ordenado em memória utilizando o Merge Sort interno, garantindo uma complexidade de $O(n \log n)$.

- Intercalação (Merge Externo):

Após todos os blocos estarem ordenados, o programa realiza uma intercalação em múltiplas etapas, combinando os blocos ordenados até formar um único arquivo final completamente ordenado.

- Geração do Relatório:

Ao final da execução, o programa cria automaticamente um arquivo relatorio.txt, contendo o tamanho total do arquivo processado, o número de blocos utilizados e o tempo total de execução (em segundos).

3.2. Complexidade:

- Tempo: $O(n \log n)$ = o mesmo comportamento do merge sort tradicional, mas adaptado para operações de disco.

- **Espaço:** $O(n)$ = o uso de espaço é distribuído entre a memória principal e o armazenamento secundário, de forma a manter o consumo de RAM dentro do limite físico disponível.

4. Análise de Complexidade

O desempenho do algoritmo foi avaliado com base no tempo total de execução registrado no relatório gerado automaticamente.

Durante os testes, observou-se que:

- A leitura e gravação em disco são as operações mais custosas, impactando diretamente o tempo total de execução;
- O tempo de ordenação em memória é significativamente menor, já que cada bloco cabe confortavelmente na RAM;
- A etapa de intercalação cresce de forma logarítmica com o número de blocos, mantendo a execução eficiente mesmo para arquivos muito grandes;
- Em média, o algoritmo apresentou um comportamento linear em relação ao volume de dados. Para arquivos de até 8 GB, o desempenho foi estável e proporcional ao aumento de tamanho, sem gargalos significativos de E/S (entrada e saída).

O relatório de execução permite avaliar de forma prática o impacto do tamanho do arquivo e do número de blocos na eficiência global do processo.

5. Descrição do Makefile e Instruções para Compilação

O projeto inclui um arquivo **Makefile** responsável por automatizar o processo de compilação, execução e limpeza do código-fonte. Essa ferramenta elimina a necessidade de compilar manualmente os arquivos, tornando o processo mais eficiente e padronizado.

A estrutura de diretórios do projeto está organizada da seguinte forma:

Projeto/

- └── src/ → contém o arquivo main.c e o cabeçalho ordenacaoExterna.h
- └── include/ → reservado para eventuais cabeçalhos auxiliares
- └── bin/ → armazena o executável gerado
- └── output/ → saída de dados e relatórios
- └── Makefile → script de automação da compilação

5.1. Instruções de Uso

1. Certifique-se de que o arquivo **Makefile** está localizado na raiz do projeto, e o código-fonte dentro da pasta **src/**.
2. No terminal, dentro da pasta do projeto, digite **make**; isso criará o executável **ordenacao_externa** dentro da pasta **bin/**.
3. Para executar o programa, digite **make run**. O programa solicitará a quantidade de gigabytes (GB) de dados a serem processados.
4. Ao final da execução, será gerado automaticamente um arquivo chamado **relatorio_execucao.txt**, contendo informações detalhadas do processo (tempo total, tamanho do arquivo, quantidade de blocos utilizados e arquivos gerados).
5. Para limpar os arquivos temporários e binários, utilize **make clean**.

6. Conclusão

O desenvolvimento deste trabalho permitiu compreender, de forma prática e aplicada, os desafios inerentes ao processamento e ordenação de grandes volumes de dados que não cabem inteiramente na memória principal.

A implementação do algoritmo de Ordenação Externa demonstrou-se eficiente e escalável, permitindo lidar com arquivos que variam de dezenas de megabytes até dezenas de gigabytes, sem sobrecarregar os recursos do sistema.

Além disso, o projeto reforçou a importância da otimização de operações de entrada e saída (I/O), evidenciando que, em contextos de grandes volumes de dados, o desempenho global do algoritmo está diretamente relacionado à forma como o programa gerencia o acesso à memória secundária.

Por fim, a integração do Makefile garantiu uma compilação automatizada, limpa e reprodutível, refletindo boas práticas de engenharia de software e consolidando o aprendizado de organização de projetos em linguagem C.